

HOTEL BOOKING SYSTEM

A PROPOSAL PRESENTED BY

K.S. WATHSALA

(S19538)

to the Department of Statistics and Computer Science of

the **FACULTY OF SCIENCE**

in partial fulfillment of the requirement

for the award of the degree of

BACHELOR OF SCIENCE

of the

UNIVERSITY OF PERADENIYA

SRI LANKA

2019

DECLARATION

I do hereby declare that the work reported in this project report/thesis was exclusively carried out by me under the supervision of Mr.Isuru Madugalla..... It describes the results of my own independent research except where due reference has been made in the text. No part of this project report/thesis has been submitted earlier or concurrently for the same or any other degree.

Date:

.....

Signature of the Candidate

Certified by:

1. Supervisor (Name): Mr.Isuru Madugalla

Date:

Signature:

2. Supervisor (Name): Dr.H.O.E. Dayarathne

Date:

Signature:

Department Stamp:

ABSTRACT

Built with modern technologies like Spring Boot for the backend, Tailwind CSS for the frontend, and MySQL for database management, the hotel booking system provides robust functionality and a user-friendly interface. The platform streamlines the booking process and simplifies room management for both customers and receptionist.

For Customers:

The Rooms Page allows customers to view room details, check availability with a date picker, and reserve rooms. After payment, customers receive a confirmation email with booking details. The Contact Us Page also enables customers to submit feedback and reviews.

For Receptionist:

The Receptionist Dashboard helps manage bookings with features such as viewing booking details, filtering by date, and managing or canceling bookings. Receptionists can also edit room information and add new rooms using an intuitive form.

This system enhances user experience and operational efficiency, providing a smooth booking experience for customers and effective room management for receptionists.

TABLE OF CONTENTS

DECLARATION	ii
ABSTRACT	iii
TABLE OF CONTENTS	iv
LIST OF TABLES	vi
LIST OF FIGURES	vii
CHAPTER 1: INTRODUCTION	1
1.1 Problem Statement	1
1.2 Gap	2
1.3 Approach	2
CHAPTER 2: LITERATURE REVIEW	
2.1 Background	4
2.2 Related Work	4
CHAPTER 3: METHODOLOGY	
3.1 Problem Definition and Scope Definition	6
3.2 Data Collection and Preprocessing	7
3.3 Data Storage	8
3.4 Backend Development	10
3.5 Frontend Development	11
3.6 Integration and Testing	11
3.7 Deployment and Maintenance	12
CHAPTER 4: IMPLEMENTATION	
4.1 Frontend	14
4.2 Backend	14

4.3 Testing and Validation	15
CHAPTER 5: RESULT DISCUSSION	
5.1 Design Decision	16
5.2 Development Time	18
5.3 Project Structure	19
5.4 Work and Team Management	20
5.5 User Interface Design	21
CHAPTER 6: CONCLUSION	
6.1 Advantages of Implemented System	29
6.2 Issues	29
6.3 Conclusion	30
CHAPTER 7: REFERENCES	31

LIST OF TABLES

Table 4.1: Implementation

LIST OF FIGURES

- Figure 3.1: ER Diagram for the project**
- Figure 3.2: Application Architecture**
- Figure 3.3: Work Breakdown Structure and Timeline**
- Figure 5.1: Frontend Project Structure**
- Figure 5.2: Backend Project Structure**
- Figure 5.3: UI for Navigation Bar of the Home Page**
- Figure 5.4: UI for Footer of the Home Page**
- Figure 5.5: UI for Check Availability of the Rooms**
- Figure 5.6: UI for Individual Rooms**
- Figure 5.7: UI for Individual Room Information**
- Figure 5.8: UI for Individual Room Booking**
- Figure 5.9: UI for Room Booking Payment**
- Figure 5.10: UI for Description of the Contact Us Page**
- Figure 5.11: UI for Rest of the Contact Us Page**
- Figure 5.12: UI for Reviewing of the Contact Us Page**
- Figure 5.13: UI for Google Map of the Contact Us Page**
- Figure 5.14: UI for Booking Details of the Receptionist Dashboard**
- Figure 5.15: UI for Room Details of the Receptionist Dashboard**
- Figure 5.16: UI for Adding Room of the Receptionist Dashboard**

CHAPTER 1

INTRODUCTION

Hotel La-Vila is a famous hotel and well-known for offering the best hospitality in the Weligama area. The hotel aims to provide customers with better services, accommodate more guests, and offer a lifetime experience as it continues to rise in popularity. Currently, all operations are conducted manually, which has resulted in unexpected issues and several complaints from customers. To address these challenges, our project involves developing a hotel booking system as a solution to the problems faced by the existing manual system.

The main requirement from the client is a user-friendly and effective management system, which includes an online reservation system, room status updates, check-in and check-out functionalities, yoga services, and payment handling. Additionally, the system will enable the management of user databases with client-permission controls.

Considering the client's requirements, we decided to design the website's front-end using React, its back-end using Spring Boot, and its database using MySQL. The new system aims to resolve existing issues while helping the hotel administration make the booking process more effective, efficient, and competitive.

This website is designed to provide guests with an easy way to plan their vacations by offering services such as online reservations, secure payments, yoga services in collaboration with instructors, and event registration. Registered users can also explore the extensive facilities provided by the hotel and search for various activities to engage in during their stay through the website's user-friendly interface.

We intend to deliver an efficient, informative, and user-friendly platform as per the client's request. Our project will focus on addressing critical internal operations, with an emphasis on the hotel's booking system. Since hotel employees are busy and lack the time to manage everything manually, this web application will significantly aid in handling the primary duties of the hotel, making operations smoother and more organized.

1.1 Problem Statement

Efficient guest interaction and day-to-day activity management are vital for delivering quality services in the hospitality industry. At present, Hotel La-Vila's operations are heavily dependent on manual processes, such as paperwork and phone-based communication, which are both time-consuming and prone to errors. The existing paper-based reservation system creates numerous inefficiencies and operational challenges, including:

- Inefficient booking workflows that lead to delays and potential customer dissatisfaction
- Disorganized and scattered data due to the absence of a centralized database
- Difficulty in tracking room availability and managing extensive customer records accurately

- Barriers to effective communication between hotel staff and external service providers
- Insufficient measures to ensure the security and confidentiality of guest information
- Limited ability to generate insightful analytical reports for informed decision-making
- Complications in payment processing and collection, causing delays and errors
- Frequent inaccuracies and incompleteness in manual record-keeping

Addressing these challenges is essential to enhance operational efficiency, improve customer satisfaction, and maintain Hotel La-Vila's reputation for exceptional hospitality.

1.2 Gap

- **Technology Gap:** The manual system lacks the integration of advanced technologies like React, Spring Boot, and MySQL, which are pivotal for modernizing operations.
- **Efficiency Gap:** Current processes are slow and prone to errors, necessitating automation to eliminate repetitive tasks and delays.
- **Customer Experience Gap:** There is no online platform for booking rooms, checking availability, or exploring hotel services, limiting guest convenience.
- **Data Management Gap:** The absence of a centralized database results in fragmented and inefficient data handling.
- **Communication Gap:** Poor coordination between staff and external service providers hampers service delivery.
- **Security Gap:** Weak security protocols in the existing system expose guest information to potential risks.

Addressing these gaps will enable the creation of an efficient, secure, and user-friendly hotel management system. This solution will enhance guest satisfaction, streamline internal processes, and position Hotel La-Vila as a leader in adopting modern hospitality practices.

1.3 Approach

To address these challenges and fulfill the identified objectives, we propose a systematic approach to developing a comprehensive Hotel Booking System for Hotel La-Vila.

1. **Analyzing Current Processes:** Conduct a thorough analysis of the existing manual workflows to pinpoint inefficiencies and bottlenecks. This step involves understanding current practices for managing reservations, tracking room availability, and maintaining guest records. Identifying these issues is critical to designing a system that replaces manual tasks with automated solutions.
2. **Designing a User-Friendly Platform:** Develop an intuitive and engaging web-based platform using React.js. This platform will provide functionalities such as room availability searches, online reservations, and access to hotel services like yoga sessions and event registration. For administrators, tools for managing bookings, updating room statuses, and handling customer data will be integrated into the platform.

3. **Implementing a Robust Back-End:** Utilize Spring Boot for the back-end system to manage business logic and API requests. A centralized MySQL database will be employed to store guest information, booking records, and payment details securely and efficiently. Real-time updates and efficient data retrieval will be key features of this back-end system.
4. **Improving Communication:** Introduce real-time updates and notification systems to enhance coordination between hotel staff and third-party service providers. These features will ensure smooth operations for services like yoga sessions and event management.
5. **Enhancing Security:** Adopt advanced security measures, including encrypted communications, secure payment gateways, and role-based access control, to protect sensitive guest data. Adherence to industry standards will further ensure data privacy and security.
6. **Adopting Agile Methodology:** Implement an iterative development process to incorporate regular feedback from stakeholders. Continuous testing will ensure the system is user-friendly, bug-free, and aligned with client requirements.

By following this approach, Hotel La-Vila will transition from a manual system to a modern, automated, and customer-focused solution. This transformation will not only resolve existing inefficiencies but also establish the hotel as a leader in hospitality technology adoption, ensuring long-term success.

CHAPTER 2

LITERATURE REVIEW

2.1 Background

Hotel booking systems have improved greatly over the years, moving from manual methods to automated and web-based solutions. Earlier systems relied on phone calls, walk-in reservations, and handwritten logs, which often caused delays, mistakes, and communication issues. As the hospitality sector expanded, there was a clear need for centralized systems to handle reservations, monitor room availability, and manage guest information more efficiently.

Modern booking systems make use of advanced technologies like web development frameworks, cloud computing, and data analytics. These systems offer real-time updates, secure payment processing, and user-friendly experiences. They not only streamline operations but also help hotels stay competitive in an industry driven by technology.

2.2 Related Work

1. Centralized Online Booking Platforms

Centralized platforms allow users to browse, compare, and book accommodations easily. They aggregate listings from multiple hotels and provide seamless payment options.

- Examples:
 - Booking.com: Offers features such as price comparisons, real-time availability, and guest reviews.
 - Agoda: Specializes in deals for last-minute bookings and international travel.

2. Integration with Payment Gateways

Secure online payment options are integral to hotel booking systems, ensuring customer trust and convenience.

- Examples:
 - Stripe: Integrated into platforms for seamless payment processing.
 - PayPal: Offers secure and fast payment methods for online bookings.

3. Cloud-Based Reservation Systems

Cloud computing enables scalability, security, and remote access, making management easier for hotels and customers alike.

- Examples:

- Cloudbeds: A property management system offering reservation, payment, and reporting features.
- Little Hotelier: Focused on small-scale accommodations, integrating booking and channel management tools.

4. User-Generated Reviews and Ratings

Reviews and ratings provide social proof and help customers make informed decisions.

- Examples:
 - TripAdvisor: Offers detailed customer reviews, photos, and activity recommendations.
 - Google Hotels: Integrates customer feedback and ratings into its search results.

5. Third-Party Service Integration

Modern systems integrate additional features like transport, events, and dining reservations to provide a comprehensive travel solution.

- Examples:
 - Trip.com: Combines hotel bookings with flight and car rental options.
 - Klook: Offers activity and attraction bookings alongside hotel reservations.

CHAPTER 3

METHODOLOGY

3.1 Problem Definition and Scope Definition

Problem Definition

The primary objective of this system is to provide a user-friendly interface that ensures a seamless user experience across all devices. Users can easily explore and book accommodations, interact with hotel activities, and complete their bookings without confusion or delays. By enhancing the ease of use and efficiency of the system, the ultimate goal is to improve guest satisfaction and enhance their overall experience.

Scope Definition

This project focuses on developing a comprehensive hotel booking system that streamlines the booking process for customers, equips hotel staff with efficient management tools, and integrates additional features to enhance the user experience. Key elements of the system include:

1. User-Friendly Booking Process
 - Customers can create secure accounts and update their details.
 - Online room bookings are based on user preferences, with real-time availability displayed.
 - Activity registration is integrated, allowing customers to sign up for hotel events like yoga sessions or other activities directly through the hotel website.
2. Management Tools for Hotel Staff
 - Managers can access tools to manage customer reviews, enabling them to address feedback and improve services.
 - The receptionist interface is equipped to register customers, manage room availability, and handle bookings efficiently.
3. Customer Reviews and Ratings
 - A review system allows guests to leave ratings and feedback about their stay.
 - These reviews help other customers make informed decisions when selecting accommodations.
4. Access Control
 - The system differentiates between registered and normal users.
 - Registered users have access to additional features such as viewing the external room views, helping them choose the most suitable room.
5. Secure Payment Gateway

- Integration with a secure payment gateway facilitates online transactions using credit cards.
 - Discounts are offered to users who make payments via credit cards.
6. Room and Amenities Details
- Information about room services and amenities is displayed, allowing users to view and select options during the booking process.
7. Enhanced Guest Experience
- Additional features like external room views enable users to make more personalized choices.
 - Integrated activity registration provides a seamless way for guests to enjoy hotel facilities beyond accommodations.

This system aims to deliver a holistic solution for both customers and hotel staff, ensuring smooth operations, enhanced customer engagement, and an improved guest experience.

3.2 Data Collection and Preprocessing

3.2.1 Data Collection

Sources: Data will be gathered from a wide range of sources, including hotel booking websites, yoga session booking platforms, APIs, and publicly available datasets. These sources provide diverse and rich data related to hospitality and tourism industries.

Data Types: The collected data will encompass various aspects, such as details about hotels, accommodations, guest houses, user reviews, ratings, pricing, availability, and additional metadata relevant to bookings.

3.2.2 Data Preprocessing

Data preprocessing is an essential step to ensure the collected data is accurate, consistent, and ready for use in the hotel booking system. This process can be divided into three key stages:

1. Data Cleaning and Formatting:

The raw data from multiple sources often contains errors, inconsistencies, and redundancies. Data cleaning involves:

- Identifying and correcting discrepancies in records.
- Removing duplicates to prevent redundancies.
- Handling missing or incomplete values by using appropriate imputation techniques or removing irrelevant entries.
- Formatting the data to maintain uniformity, such as standardizing date formats, currency types, and text case.

2. Data Normalization:

Data normalization ensures consistency across different datasets by transforming information into a standardized format. For example:

- Converting numerical fields (like prices or ratings) to a uniform scale.
- Aligning categorical data (e.g., room types or customer preferences) to avoid mismatches.

This step facilitates easier integration and improves the performance of analytical tools, including machine learning models.

3. Data Integration:

To build a cohesive hotel booking system, data from disparate sources such as room availability, customer profiles, payment records, and booking histories will be combined into a unified database. Integration ensures:

- Seamless accessibility of comprehensive data for analysis and reporting.
- Real-time updates to improve user experience and operational efficiency.
- A structured and consistent format that forms the backbone of the system's functionality.

By implementing a thorough data preprocessing strategy, the hotel booking system will ensure reliable insights, optimize database performance, and enhance both staff operations and guest satisfaction.

3.3 Data Storage

The hotel booking platform, which incorporates yoga services alongside other offerings, will employ a Relational Database Management System (RDBMS) to manage its data efficiently. A carefully designed database schema will organize core entities such as guest profiles, room inventory, booking details, payment transactions, yoga session schedules, and metadata for hotel services. This structured design will enable smooth integration between various components of the system.

The database will adhere to normalization principles to maintain data integrity and minimize redundancy. This approach will eliminate duplicate entries and ensure consistent relationships between tables, improving both storage efficiency and data reliability.

Performance will be enhanced through techniques such as indexing and query optimization:

- **Indexing:** Frequently accessed fields, such as room availability or yoga schedules, will be indexed to enable faster retrieval and improve response times for user queries.
- **Query Optimization:** Complex queries will be optimized to handle high-volume operations efficiently, ensuring system responsiveness during peak periods.

The database architecture will be designed for scalability, allowing the system to adapt to increasing data volumes or expanded functionality in the future. By implementing these strategies, the data storage framework will provide a robust, reliable, and high-performing

foundation for the hotel booking platform, ensuring operational efficiency and a seamless user experience.

ER Diagram for the project

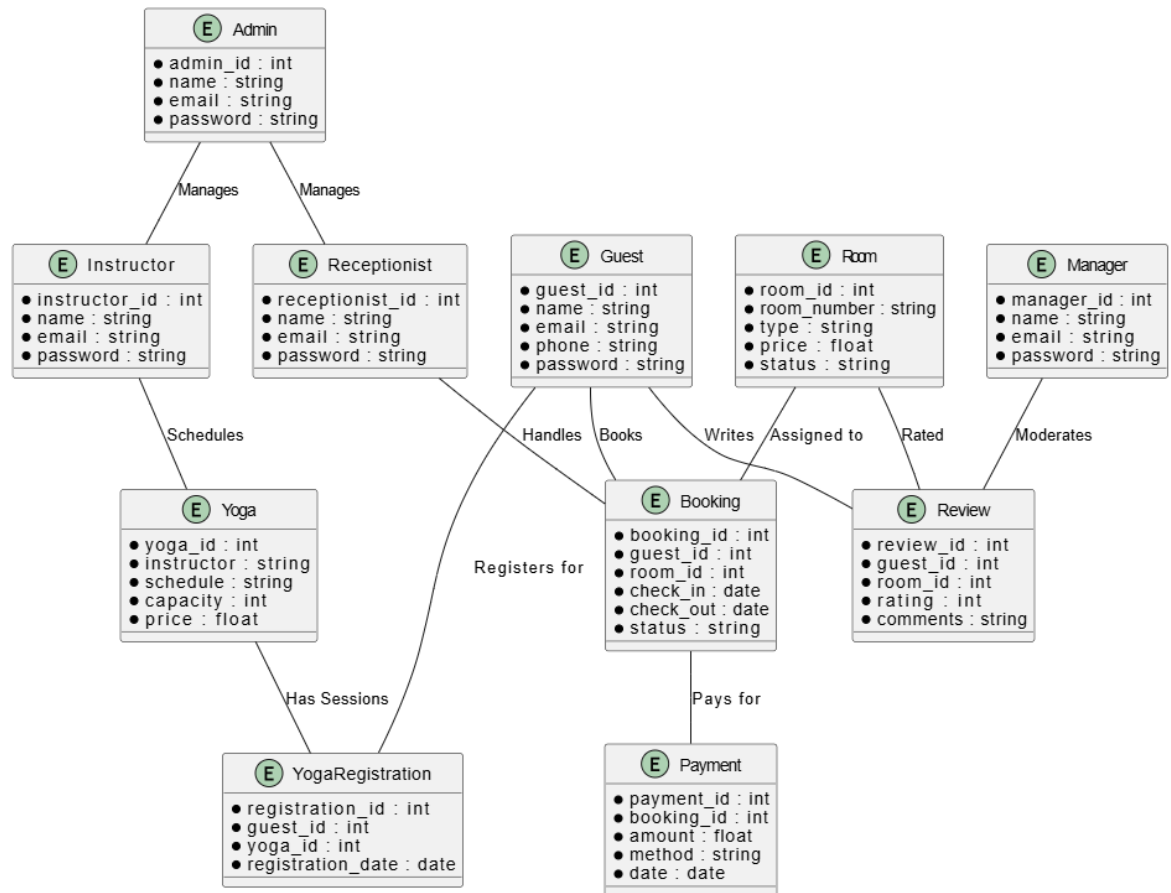


Figure 3.1

Application Architecture

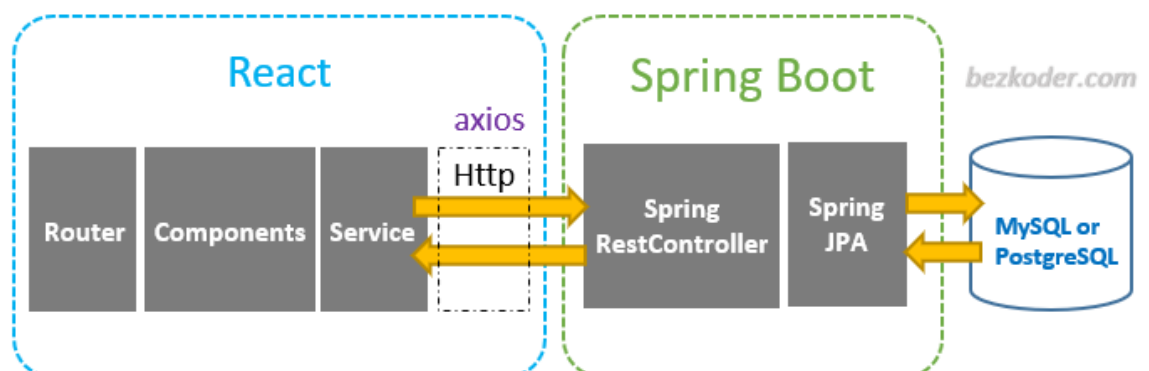


Figure 3.2

3.4 Backend Development

Initialize Spring Boot Project

- Set up the project using Spring Boot initializer.
- Include dependencies for Spring Web, Spring Data JPA, Spring Security, and MySQL Driver.

Install Core Dependencies

- Hibernate (ORM for database).
- Database drivers (MySQL, PostgreSQL as needed).

Modules

✓ Room Module (As a customer)

Entity: Define room details (room type, availability, pricing).

DTO: Handle room creation and updates.

Service: Business logic for room availability and bookings.

Controller: Expose APIs for room-related operations.

✓ Booking Module (As a customer)

Entity: bookings (guest ID, room ID, dates, status).

DTO: Manage booking requests and responses.

Service: Handle booking logic, including payment processing.

Controller: Expose endpoints for booking rooms.

✓ Payment Module (As a customer)

Entity: Define payment details (amount, method, date).

Service: Integrate with payment gateways and manage transactions.

Controller: Handle payment endpoints.

✓ Review Module (As a customer)

Entity: Put customer reviews (rating, comments, room ID).

DTO: Handle review data.

Service: Manage and retrieve reviews.

Controller: Expose APIs for review management.

Configuration

Database-Configure MySQL database using application.properties file

Role-based authorization for different user types.

Validation-Add validation for inputs using Hibernate Validator.

Utilities

Email Service(SMTP)-Send booking confirmation and notifications.

3.5 Frontend Development

The Hotel Booking System frontend is built using React.js for dynamic user interfaces and Tailwind CSS for modern, responsive styling. It incorporates reusable components such as Navbars, Footers, and forms like Signup and LoginForm to ensure a consistent and efficient design. React Router manages navigation, supporting client-side routing, nested routes, and dynamic path handling.

As the developer of the Home page, Rooms page, Contact us page and Receptionist Dashboard, my contributions include: Tools like **Vite** optimize the development process, while **Axios** and **Firebase** ensure efficient API communication and secure user authentication. My focus was on creating a scalable, responsive, and user-friendly frontend tailored to both customers and receptionists.

3.6 Integration and Testing

3.6.1 Integration

The Hotel Booking System ensures seamless integration between the frontend and backend components to enable smooth communication and robust functionality throughout the application.

- **API Integration:** Axios.js is utilized for efficient handling of HTTP requests, enabling the frontend to interact seamlessly with the backend.
- **Real-Time Updates:** If required, Firebase can be incorporated for managing real-time updates and notifications. Additionally, it could facilitate features such as user authentication and push notifications.
- **CORS Configuration:** Configured CORS policies to enable secure and authorized communication between the React.js frontend and Spring Boot backend running on different domains or ports.

3.6.2 Testing

- **Integration Testing:** Ensure smooth communication between the frontend, backend, and modules, including customer booking functionality and receptionist management.
- **End-to-End (E2E) Testing:** Test complete workflows, such as customers making bookings and receptionists managing room assignments.
- **Performance Testing:** Evaluate the system's performance, ensuring it handles high traffic efficiently.
- **Security Testing:** Ensure sensitive customer and booking data is securely encrypted.

- **API Testing:** Test API interactions, ensuring correct behavior for customer bookings and receptionist tasks.

3.7 Deployment and Maintenance

3.7.1 Deployment

The system will be deployed on a local server for initial testing and usage. Continuous Integration/Continuous Deployment (CI/CD) pipelines will be set up to automate the deployment process, ensuring efficient and streamlined updates to the application.

3.7.2 Maintenance

- **Monitoring and Maintenance:** Ongoing monitoring will be carried out to identify and resolve performance issues or anomalies in real-time.
- **Updates and Enhancements:** Regular system updates will be implemented to improve functionality, user experience, and overall system stability. These updates will focus on enhancing features and addressing any emerging issues.

Work Breakdown Structure and Timeline

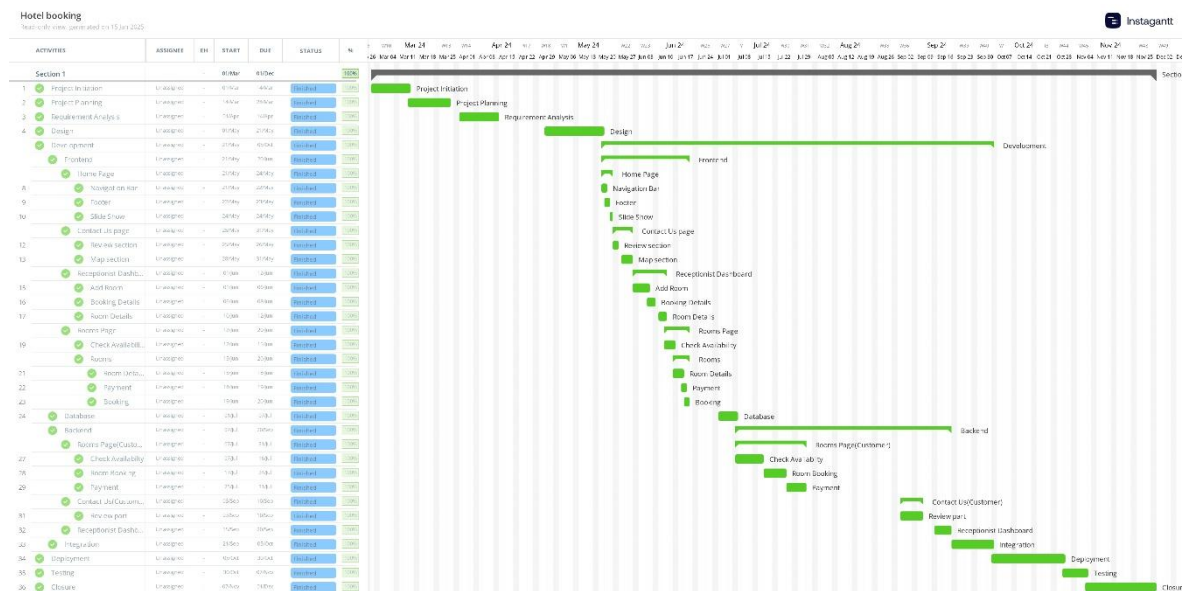


Figure 3.3

CHAPTER 4

IMPLEMENTATION

Aspect	Details
Homepage	Displays a welcoming interface with options for navigating to rooms, bookings, and contact information
Rooms Page	Room Management View Rooms: - Customers can explore a detailed list of available rooms with descriptions, photos, amenities, and pricing Check Room Availability: - Using the check-in and check-out date picker, customers can instantly verify room availability Booking Flow Book a Room: - Customers can reserve a room by filling out the booking form and proceeding to payment. The room is held temporarily until the payment is successfully completed Payment and Confirmation Email: - After payment, customers receive an email with their booking confirmation, check-in and check-out details, and a unique reference number
Contact Us Page	Submit Reviews: - Customers can submit their reviews directly from the "Contact Us" page to provide feedback Google Map: - The Google Map embedded on the Contact Us page provides an interactive visualization of the location, "Villa The Leaf Waligama"
Receptionist Dashboard	Booking Details - Displays a calendar to filter bookings by date. - Shows a list of bookings with details such as: - Booking ID - Room ID - Check-In and Check-Out dates - Guest details (Name, Email) - Number of Adults and Children - Total Guests - Confirmation Status - Actions (e.g., manage or cancel bookings) Room Details - Displays a list of all available rooms with the following information: - Room ID - Room Name - Room Type - Room Price - Provides options for: - Viewing and editing room details. - Deleting a room Add Room - A form to add new rooms with fields for: - Room Name - Room Type (dropdown selection) - Fees per night - Maximum occupancy count - Description - Image URLs
Frontend Details	
Framework	React JS
Styling	Tailwind CSS

Aspect	Details
Features	User authentication Receptionist dashboard for managing bookings and rooms User-friendly booking forms for customers
Libraries	Axios (for API communication with the backend) React Router (for client-side routing)
Backend Details	
Framework	Spring Boot (Java-based backend framework for building REST APIs)
Features	REST APIs for user authentication CRUD operations for rooms and bookings Use SMTP service for email notifications
Security	Spring Security Bcrypt passwords CORS enabled
Database Details	
Database Engine	MySQL
Schema Design	Tables for Users, Rooms, Bookings, Reviews (booking, booking_pending, booking_pending_seq, booking_seq, card, review, review_seq, room, room_seq, user)
Version Control	GitHub
UI	Figma
Testing	Postman

Table 4.1

4.3 Testing and Validation

- Rooms Page Testing: Verified correct room descriptions, photos, amenities, and pricing. Tested room availability check with different dates for accuracy.
- Booking Flow Testing: Ensured customers could reserve rooms, complete payment, and receive confirmation emails with correct details.
- Contact Us Page Testing: Checked functionality for customers submitting reviews successfully.
- Receptionist Dashboard Testing: Validated correct booking information display, including date filtering and booking management.
- Room Details Testing: Ensured accurate display of room information and proper editing and deleting functionality.
- Add Room Functionality Testing: Verified that all fields (room name, type, price, description, and image URLs) were correctly saved and displayed when adding new rooms.

CHAPTER 5

RESULT DISCUSSION

5.1 Design Decision

Home Page

- Design Choice:
 - Create a welcoming interface that is visually appealing and provides easy navigation to key sections, such as Home, Rooms, Activities, Offers, Facilities Contact Us, Login/Join and SignUp.
 - Use React JS components to ensure the homepage is responsive and dynamic.

Rooms Page

1. View Rooms:
 - Display a detailed list of available rooms using React components for modularity and reusability.
 - Include rich content such as descriptions, photos, amenities, and pricing for each room.
2. Check Room Availability:
 - Integrate a date picker component for selecting check-in and check-out dates.
 - Use **real-time availability checking** by querying the database through REST APIs provided by the Spring Boot backend.

Booking Flow

1. Book a Room:
 - Implement a booking form with fields dynamically validated using React hooks.
 - Reserve the room temporarily until payment is completed using backend logic and database transactions.
2. Payment and Confirmation Email:
 - Implement payment processing using a secure payment gateway like Stripe or PayPal.

- Use the Spring Boot backend to generate and send email confirmations via SMTP integration, including details like the booking reference number, check-in, and check-out details.

Contact Us Page

1. Submit Reviews:

- Add a feedback form with real-time validation using React.
- Use the backend to store customer reviews securely in a dedicated database table.
- Display submitted reviews dynamically using React components and REST APIs.

Receptionist Dashboard

1. Booking Details:

- Use a calendar view component to filter bookings by date.
- Display booking details such as Booking ID, Room ID, Guest Details, and Confirmation Status using a dynamic data table with sorting and filtering options.
- Provide "Actions" (e.g:-Cancel Bookings) with role-based access control using Spring Security.

2. Room Details:

- Create a detailed list of available rooms with options to view, edit, or delete rooms.
- Use React modal components for editing and confirmation dialogs for deletions.
- Synchronize room details in real-time with the MySQL database through Spring Boot APIs.

3. Add Room:

- Implement a form for adding new rooms with fields for Room Name, Type, Fees, Occupancy, Description, and Image URLs.
- Use file upload components for handling images and store them securely the database.

Technology Choices

- Frontend:

- React JS for creating a dynamic and responsive UI.
- Tailwind CSS for consistent, utility-first styling.
- React Router for seamless navigation across pages.
- Backend:
 - Spring Boot for a secure and scalable REST API framework.
 - Spring Security for Role-Based Access Control (RBAC) to differentiate between receptionist, guest, and admin permissions.
 - SMTP integration for automated email notifications.
- Database:
 - MySQL for structured data storage with indexing and normalization for optimized performance.

Security Considerations

- Use bcrypt for password encryption.
- Enforce CORS policies to manage cross-origin requests securely.
- Secure sensitive data transmission through HTTPS and implement input validation to prevent SQL Injection.

Scalability

- Adopt modular design principles to allow independent scaling of features like booking, room, and review systems.

5.2 Development Time

1. Technologies Used:

- Backend: Spring Boot
- Frontend: Tailwind CSS
- Database: MySQL

2. Challenges Faced:

- Steep learning curve due to unfamiliarity with the technologies.
- Designing a user-friendly interface for managing various processes.
- Understanding and contributing to backend development using Spring Boot, which was entirely new to me.

3. Skills Gained:

- Improved coding and problem-solving abilities.
- Learned to write cleaner, more readable code while developing the frontend.
- Gained foundational knowledge in backend development using Spring Boot.

4. Collaboration and Teamwork:

- Worked closely with group members to overcome challenges.
- Received support and guidance from teammates, especially for backend development.
- Contributed to creating a standard-level frontend design.

5. Overall Impact:

- Enhanced technical skills across frontend, backend, and database integration.
- Developed a deeper appreciation for teamwork and collaboration in software development.
- Successfully completed the project by learning and adapting to new technologies.

5.3 Project Structure

Frontend

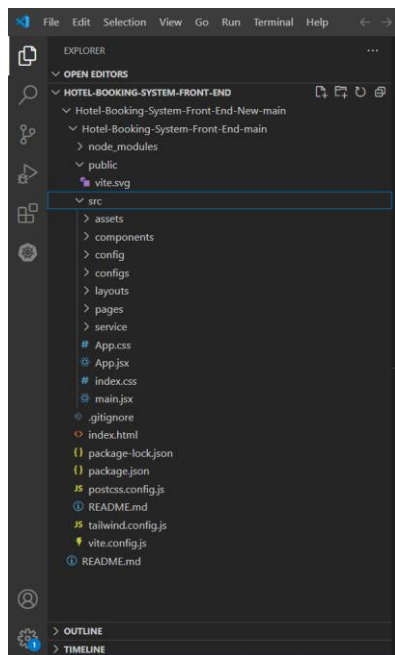


Figure 5.1

Backend

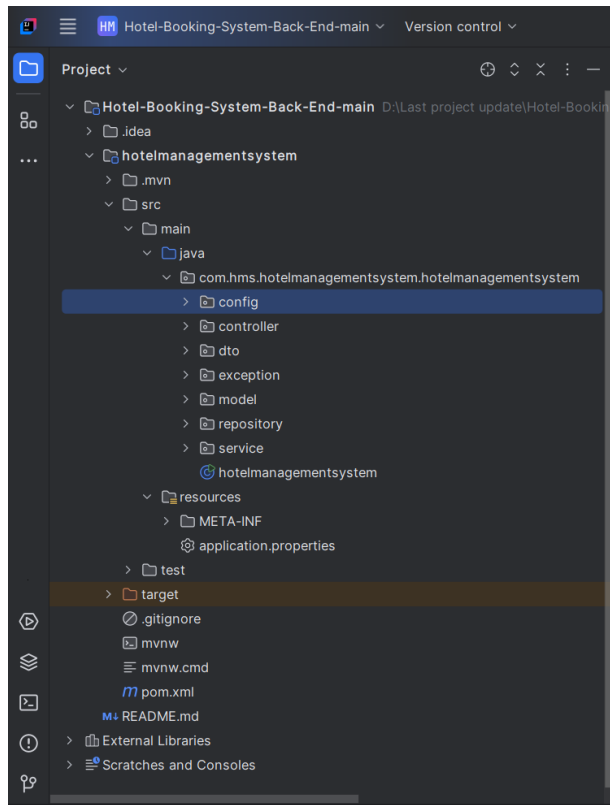


Figure 5.2

5.4 Work and Team Management

1. Team Collaboration:

- The project was a team effort, where all members actively contributed to different aspects of development.
- Faced various challenges with the unfamiliar technologies but resolved them collectively through discussions and teamwork.

2. Use of GitHub:

- Utilized GitHub as the primary tool for team and work management.
- Enabled seamless collaboration by integrating individual implementations into a unified project.
- Leveraged GitHub for version control, allowing members to make changes, review each other's work, and fetch updates efficiently.

3. Challenges in Efficiency:

- Acknowledged that work efficiency was somewhat lower compared to other teams due to limited prior experience with the frameworks.
- Despite this, the team adapted to the challenges by learning together and improving gradually throughout the project.

4. Overcoming Difficulties:

- Tackled technical and coordination issues with persistence and mutual support.
- Ensured everyone had the opportunity to contribute and grow, fostering a positive and collaborative team environment.

5. Project Outcome:

- Successfully developed the hotel booking web site.
- Gained valuable experience in both technical and teamwork aspects of software development.

5.5 User Interface Designs

- Designed unique, user-friendly interfaces tailored for each type of user in the hotel booking system.
- Ensured seamless integration of services into the main website while maintaining consistent layouts, fonts, colors, and components across the platform.
- Collaborated with team members to ensure compatibility and uniformity across all user interfaces.
- Contributed specifically to the development of the following interfaces:
 - Home page
 - Receptionist dashboard
 - Rooms page for customer

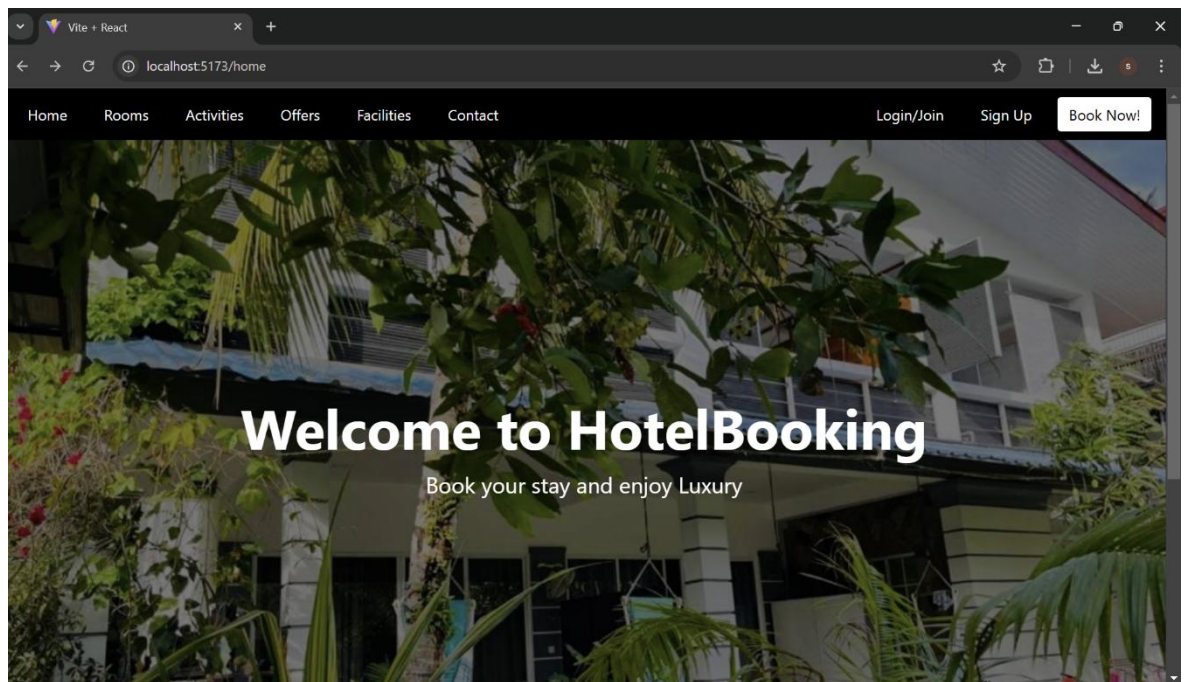


Figure 5.3

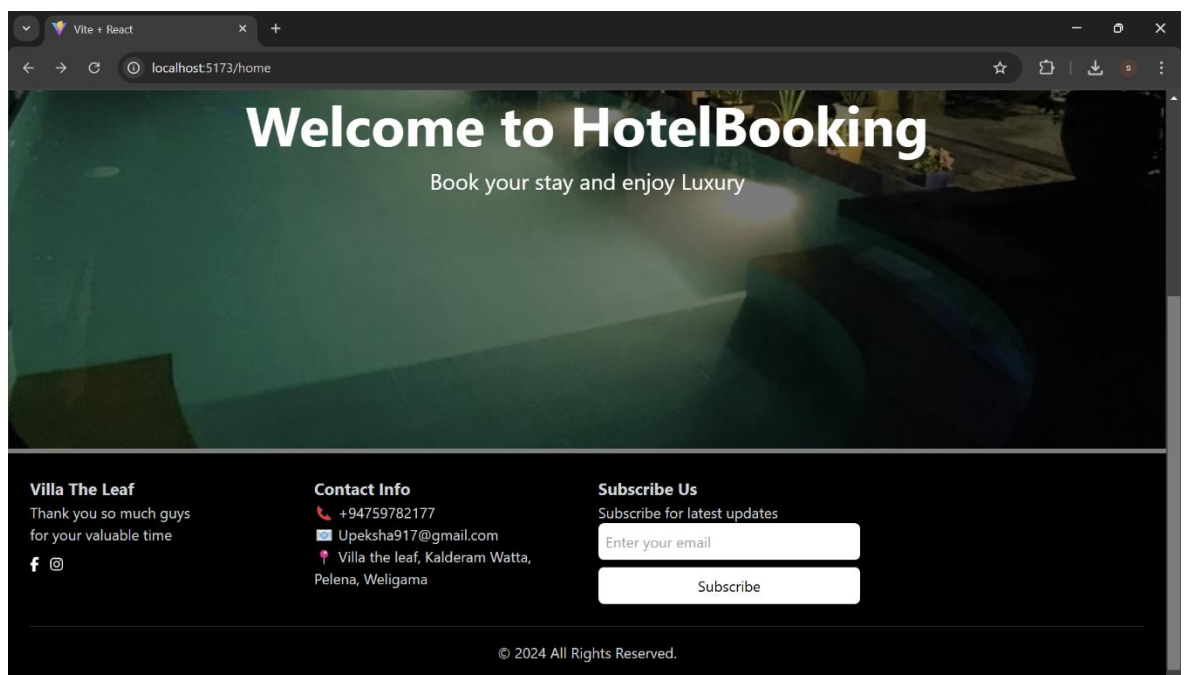


Figure 5.4

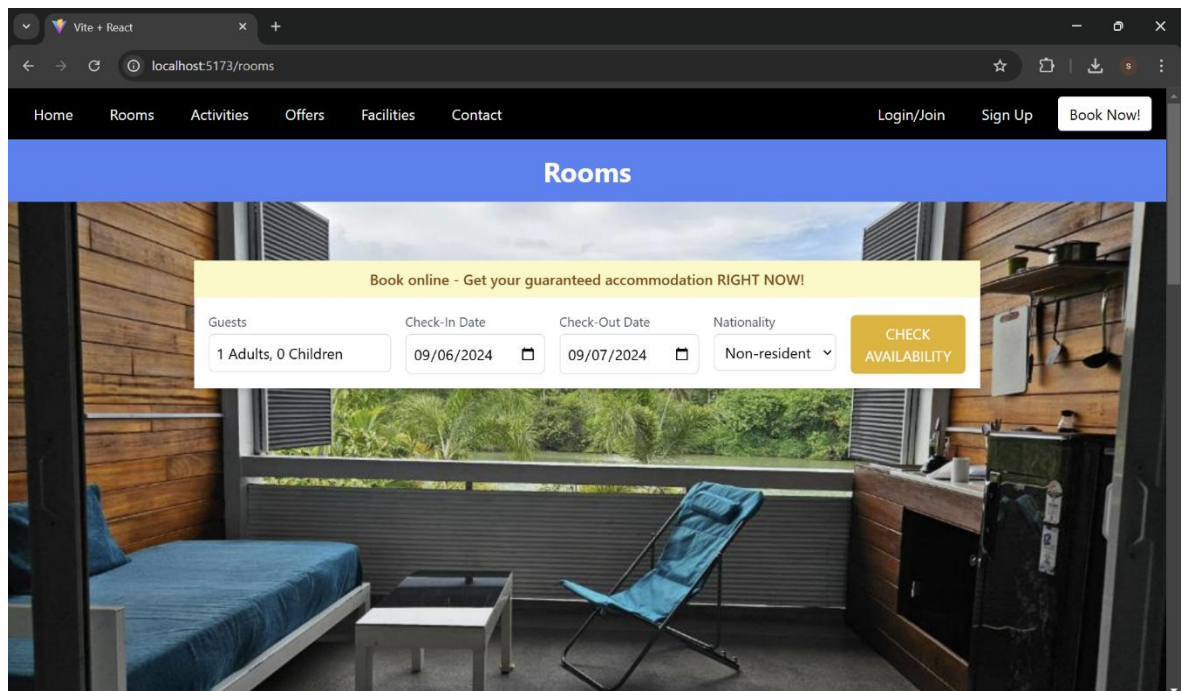


Figure 5.5

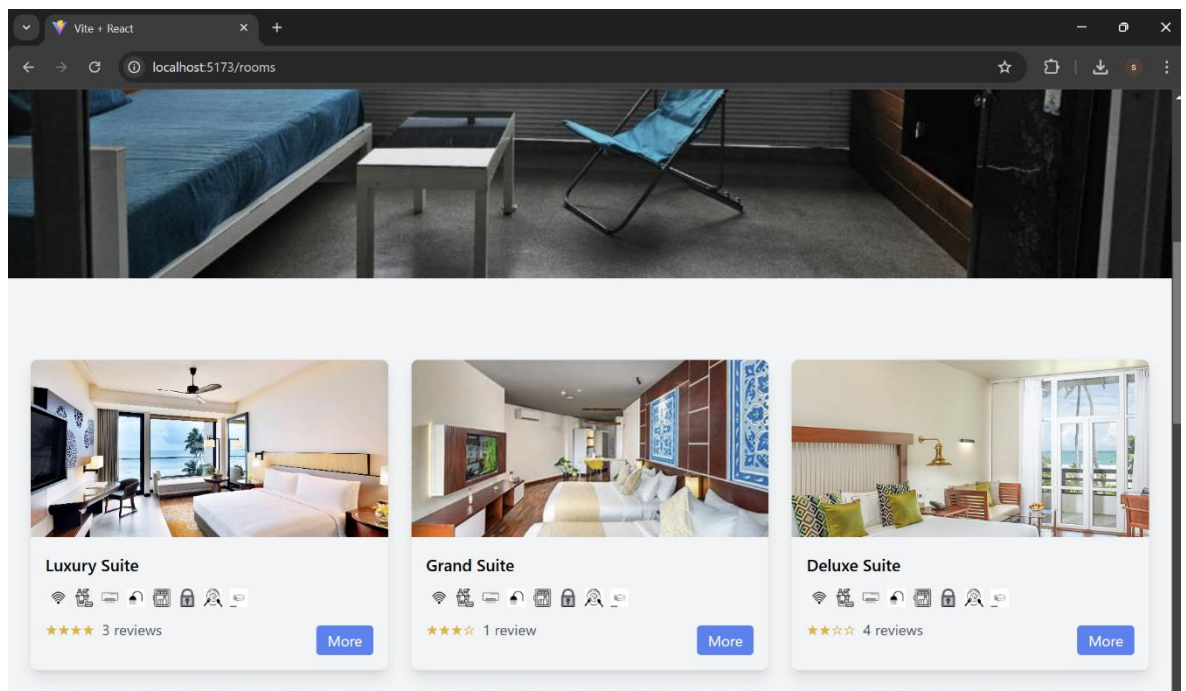


Figure 5.6

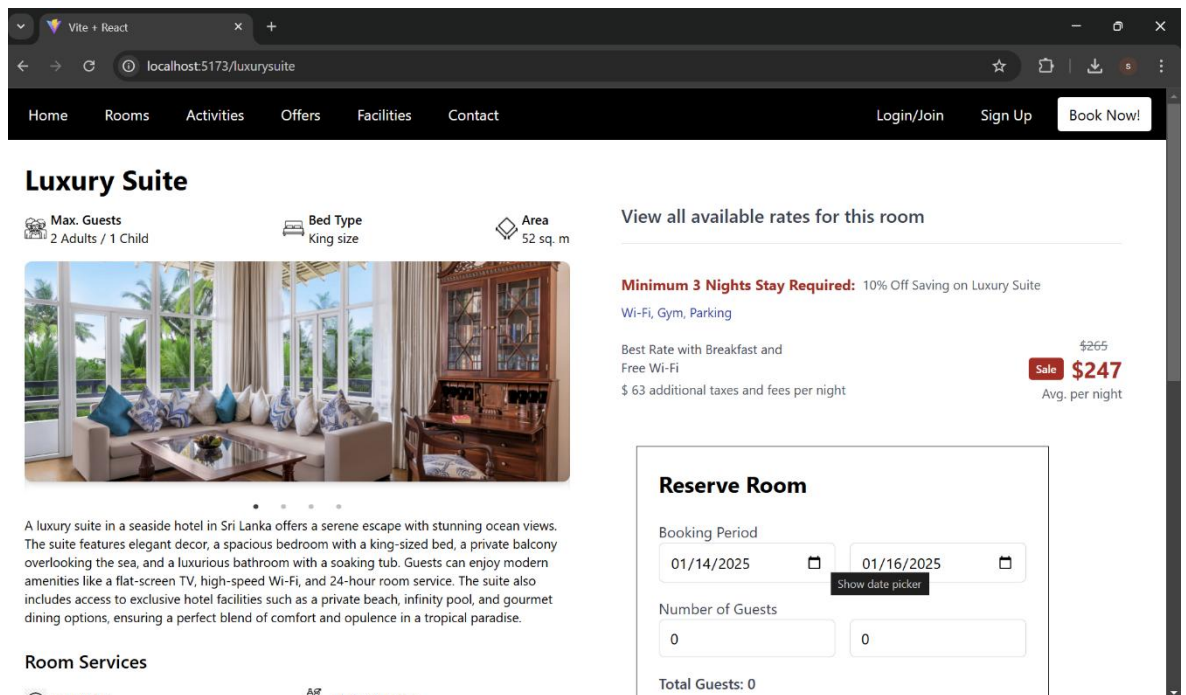


Figure 5.7

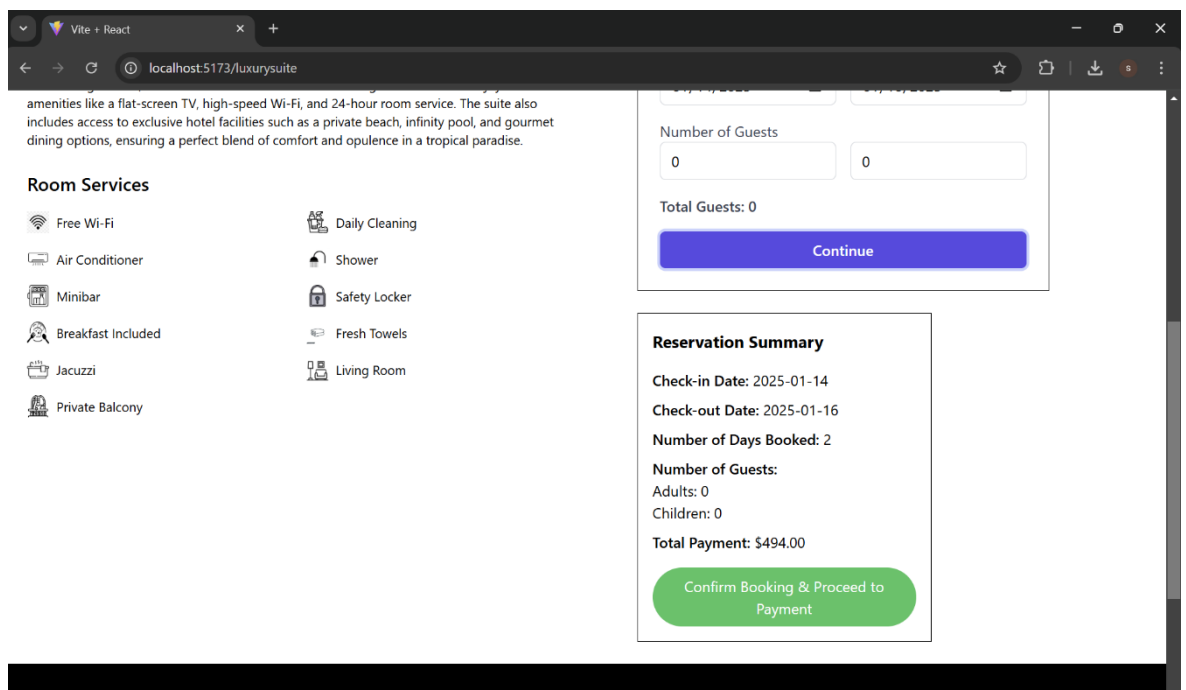


Figure 5.8

1 Your Information

First Name
Enter your first name

Last Name
Enter your last name

Email
Enter your email

Phone
Enter your phone number

Address
Enter your address

City
Enter your city

State/Country
Enter your state/country

Postcode
Enter your postcode

Additional Notes
Any additional requests

2 PAYMENT INFORMATION

Card Type
Enter card type

Card Number
Enter card number

Card Holder
Enter card holder name

CVC
Enter CVC

Expiry Month
MM

Expiry Year
YYYY

☐ I agree to the terms and conditions

Proceed to Pay

Figure 5.9

WE LOVE TO HEAR FROM YOU

Your hospitality is our priority, Luxury at its finest.

Hotel La-Vila is a famous hotel and well-known hotel with the best hospitality in Waligama area. We expect to give our guests a better service, to accommodate more guests & give a lifetime experience as a rising hotel.

We hope our hotel will give chance to guests to plan their vacation in an effective way by providing numerous facilities

Figure 5.10

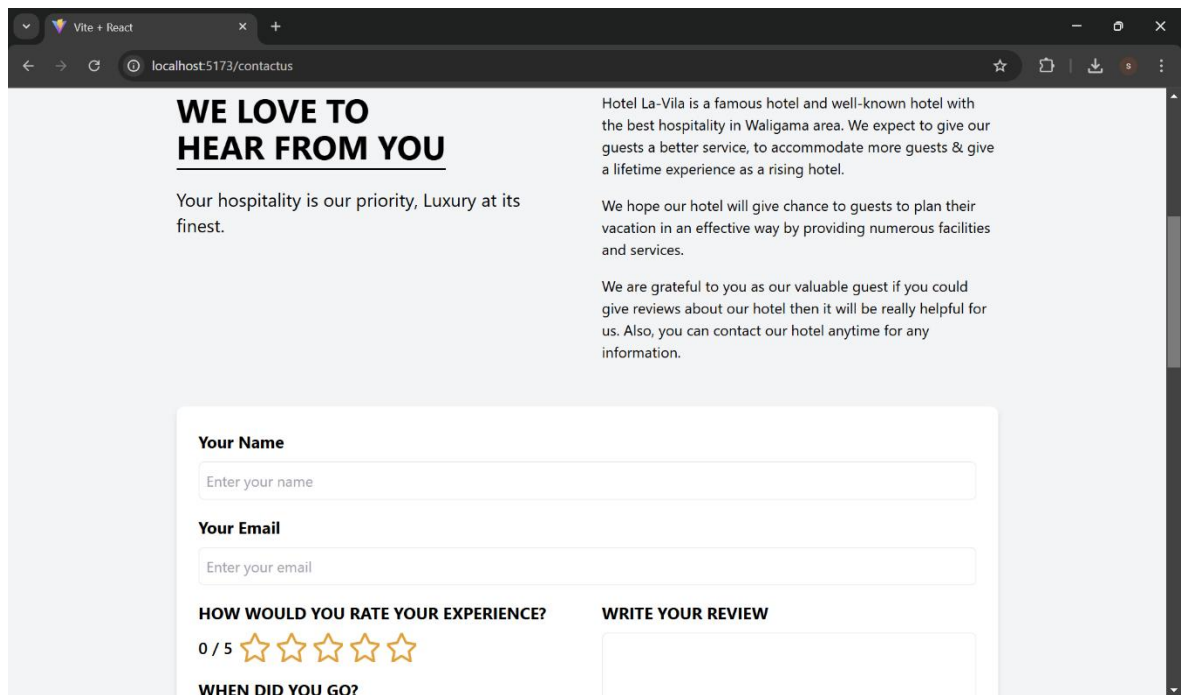


Figure 5.11

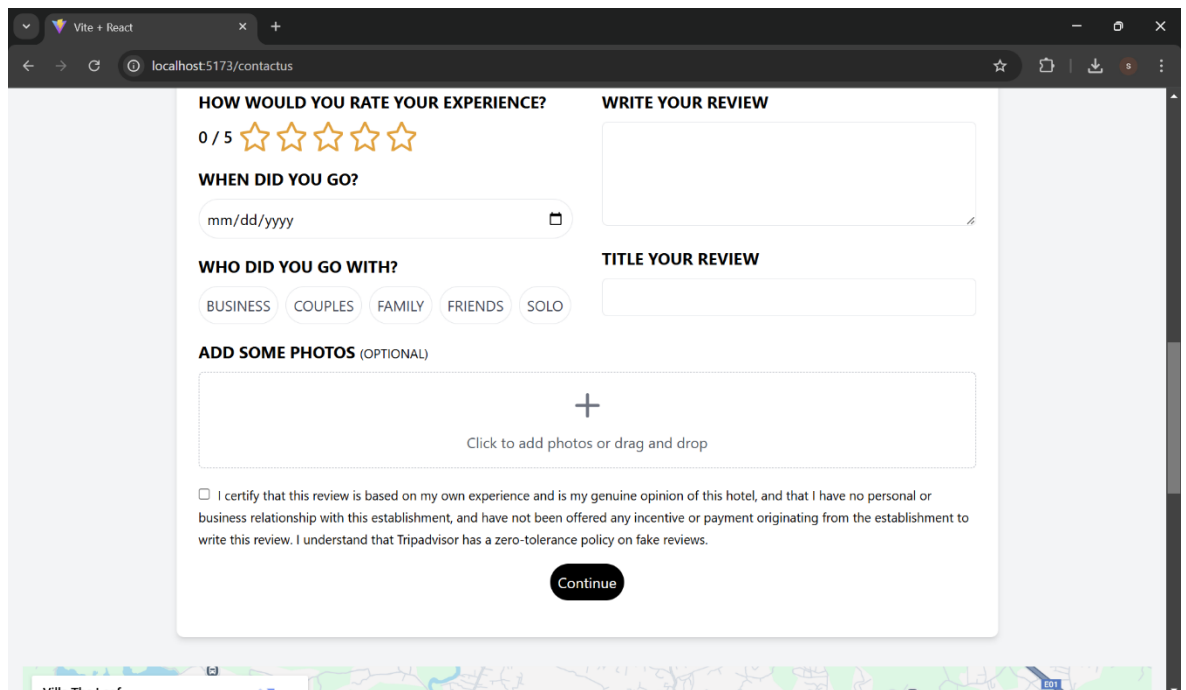


Figure 5.12

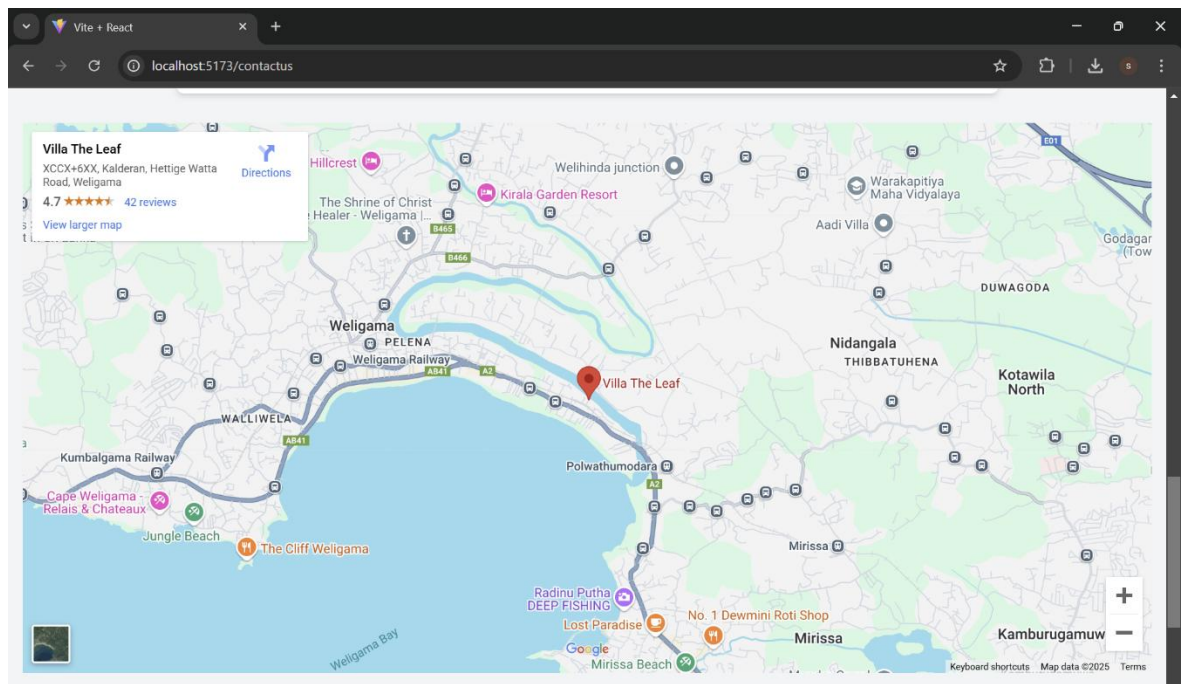


Figure 5.13

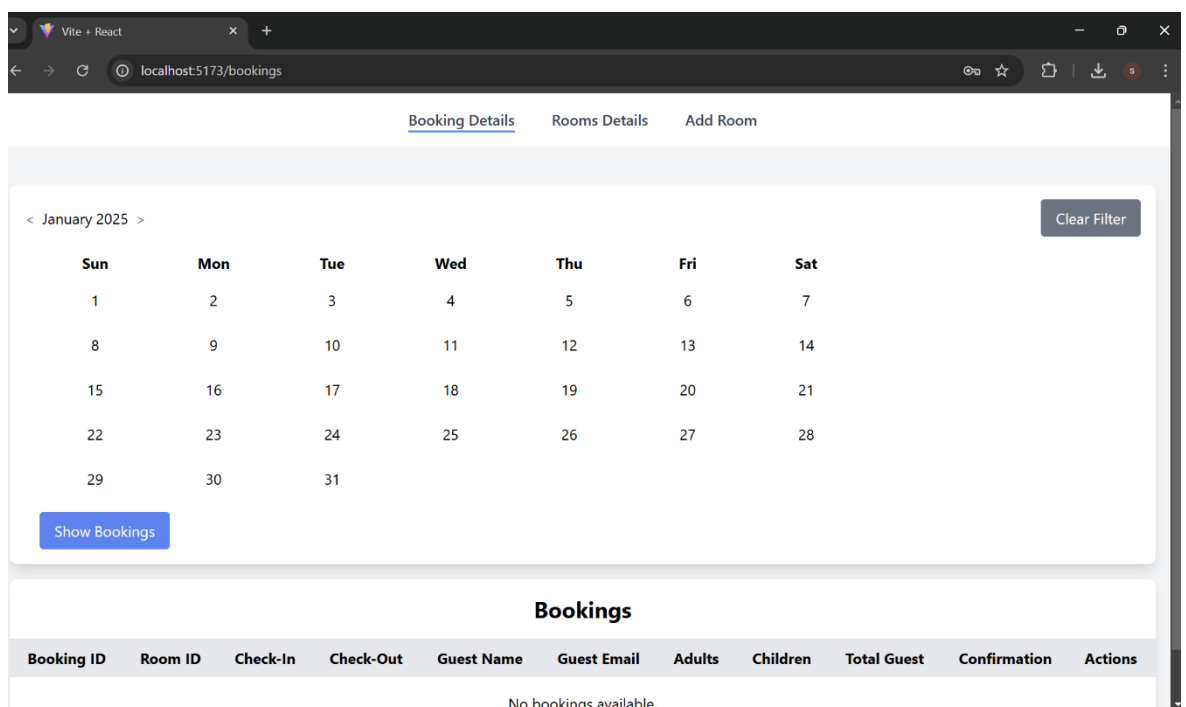


Figure 5.14

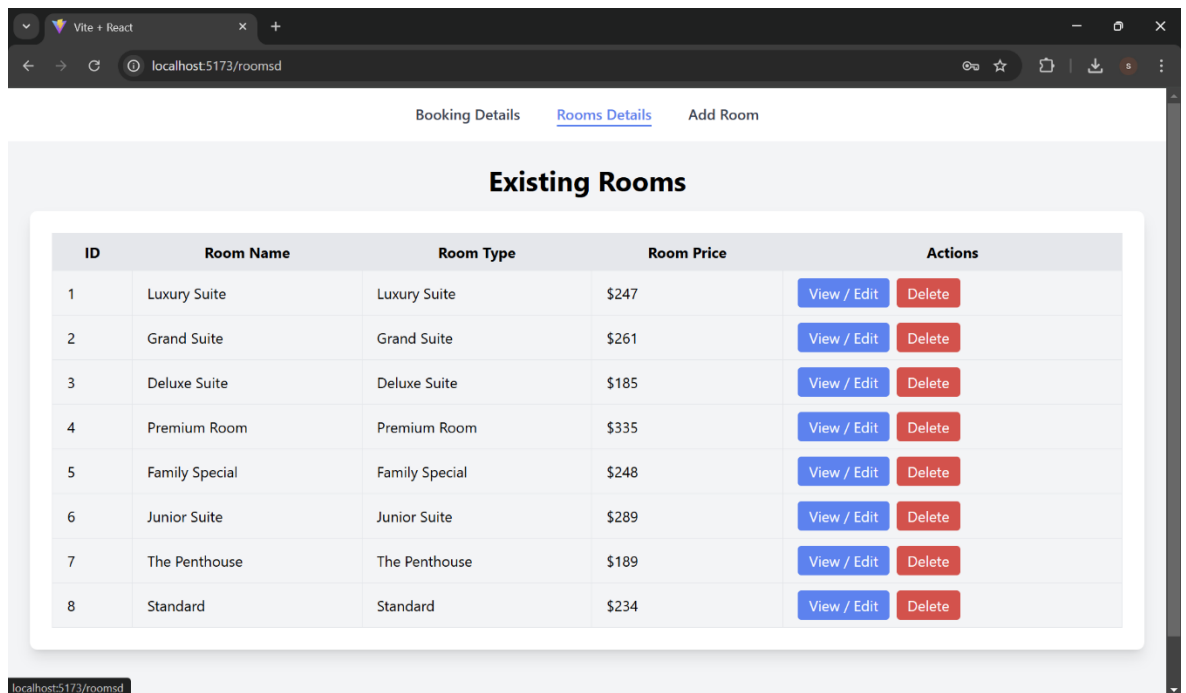


Figure 5.15

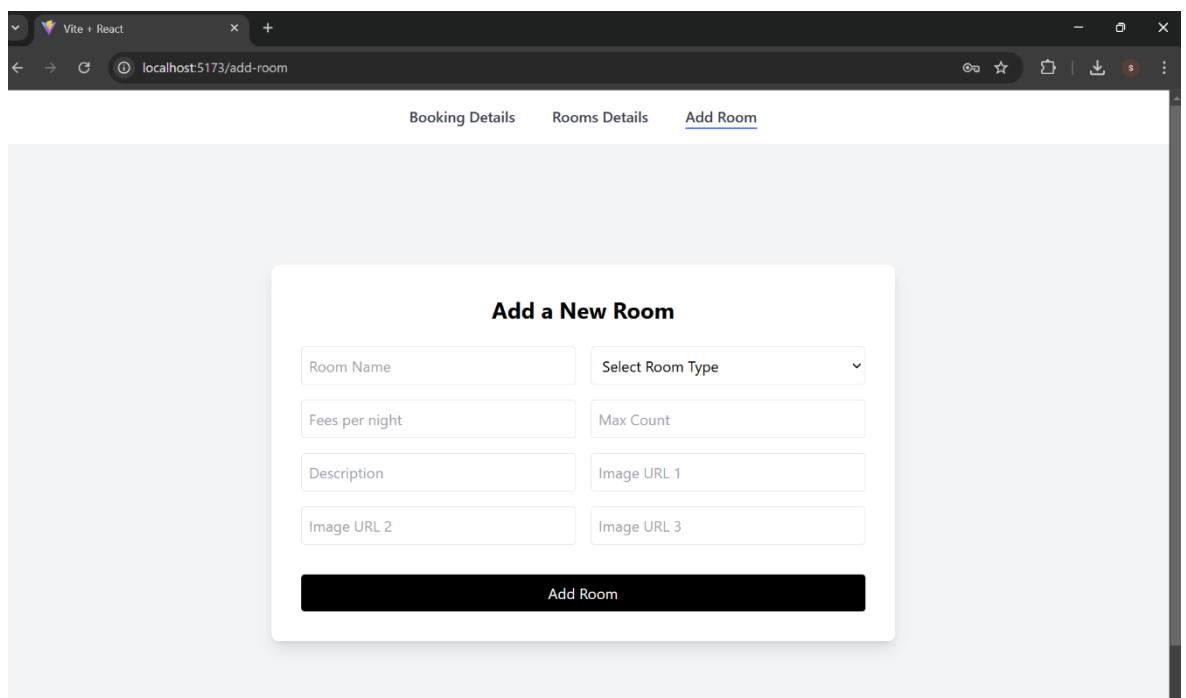


Figure 5

CHAPTER 6

CONCLUSION

6.1 Advantages of Implemented System

- **Better User Experience:** The system is easy to use for customers, administrators, and yoga instructors, making navigation simple and interactions smooth.
- **Centralized Management:** Administrators can manage hotel bookings, yoga sessions, and user accounts in one place, making operations easier and faster.
- **Support for Multiple Users:** Different users, like customers, administrators, and yoga instructors, have features designed specifically for their needs.
- **Easy Accessibility:** Customers can book hotel rooms and yoga sessions online without needing to visit or call, saving time and effort.
- **Expandable Design:** The system is built to grow, so more services, features, or user roles can be added in the future.
- **Saves Time and Money:** Automating bookings reduces the work for staff and lowers costs, benefiting both the hotel and its customers.
- **Flexible and Customizable:** The system can be adjusted to meet different needs, making it useful for a variety of hotels and services.

6.2 Issues

- **Login Issues:** Multi-user login systems can face security problems if features like password protection or session management are not strong. We also had trouble using JWT authentication because of errors when generating tokens.
- **Room Image Storage:** We tried using AWS S3 Bucket to store room images, but errors forced us to use Google image links instead.
- **Security Concerns:** Without strong security, the system could face risks like data breaches, hacking, or unauthorized access.
- **Payment Problems:** If we add online payments in the future, there could be issues like payment failures, wrong charges, or limited support for different payment methods.

6.3 Conclusion

Through my contributions to the development of the hotel booking system, particularly in the receptionist and customer interfaces, I have gained valuable experience in building user-centric, efficient systems. By focusing on both the management side for receptionists and the booking flow for customers, I have helped ensure that both user roles can easily navigate and interact with the platform.

The receptionist dashboard, which I contributed to, provides an intuitive way for managing bookings, rooms, and customer interactions. Receptionists can efficiently handle bookings, add new rooms, and maintain up-to-date room information, helping the hotel run smoothly. On the customer side, I developed features that make room browsing, booking, and payment processes seamless, ensuring that customers have a simple and pleasant experience from start to finish.

Overall, this project has not only improved my technical skills but also deepened my understanding of how web development can be used to create systems that enhance both operational efficiency and user experience. The hotel booking system is a perfect example of how technology can simplify tasks, improve communication, and provide convenient solutions to users. As the system continues to evolve, I believe it will further revolutionize the way hotel bookings and customer interactions are handled, offering both convenience and reliability.

CHAPTER 7

REFERENCES

1. General References

GeeksforGeeks: <https://www.geeksforgeeks.org/>

W3Schools: <https://www.w3schools.com/>

Baeldung: <https://www.baeldung.com/>

2. Database

MySQL: The database system used for storing user data, booking information, and other relevant records.

Documentation: <https://dev.mysql.com/doc/>

3. Authentication and Security

Spring Security: For implementing secure authentication and authorization mechanisms.

Documentation: <https://spring.io/guides/gs/securing-web/>